

PTV Delay Predictor

Introduction-

Public transport delays influence the reliability of a service, affecting travel times for commuters whilst also potentially being a cause of frustration. Accurate predictions of delays before they occur could help commuters and reduce unexpected disruptions.

This project investigates whether it is possible to predict short term train delays on the PTV metro system by using real time GTFS trip updates, weather conditions, active service alerts and delay patterns. The aim is to build a machine learning pipeline that is able to estimate how likely a delay is to happen using information that is publicly available.

This project involved developing an end to end data pipeline containing automated data collection, cloud deployment, data preprocessing, feature engineering, machine learning evaluation and model cross validation.

Methods-

Data Collection-

A python data collection service was developed and implemented on Railway, a cloud based hosting platform, to continuously collect PTV data. This occurred every 15 minutes, where the service requested information from

- PTV GTFS - realtime API for trip updates across the metro network extracting actual arrival times vs scheduled arrival/ departure times for every active trip and stop
- PTV GTFS - realtime service alerts to capture number of active network alerts at that moment
- OpenMeteo weather API for current weather conditions in Melbourne (temperature, windspeed, precipitation)

A lightweight Flask web server ran alongside the collection service as a health check endpoint and enabled retrieval of the dataset.

Over 14 days the pipeline gathered over 2.25 million records covering 16 metro routes and 536 unique stops. During deployment there was an issue with volume storage capacity, this was resolved by downloading and archiving the accumulated data and clearing the volume, allowing further data to be collected uninterrupted. This problem was fixed and automated to rotate output files improving scalability to longer data collection periods.

Exploratory data analysis-

Exploratory data analysis was performed in order to understand key characteristics about the dataset and identify issues required in preprocessing. The final dataset contained 2,252,749 records covering 14 days having 16 unique routes and 536 unique stops. From those records 2,193,578 (97.37%) were not delayed while 59,171 (2.63%) were delayed, demonstrating a highly imbalanced class distribution that informed the modelling and evaluation approaches used later in the project.

Initial analysis also showed `has_network_alert` a field which indicated whether there was an active alert at the time, had a constant value of 1 across 2.25 million records. Further investigation showed that at any time the PTV network had at least 1 alert active anywhere across its 16 routes meaning this feature had no weight.

Furthermore, there were 4411 missing weather records (temperature, windspeed, weather_code, precipitation) as well as a small number of outliers in delay times being several hours ahead or behind. These were inconsistent with realistic delays and were removed before modelling only keeping values inside a realistic range (-30 to 120 minutes).

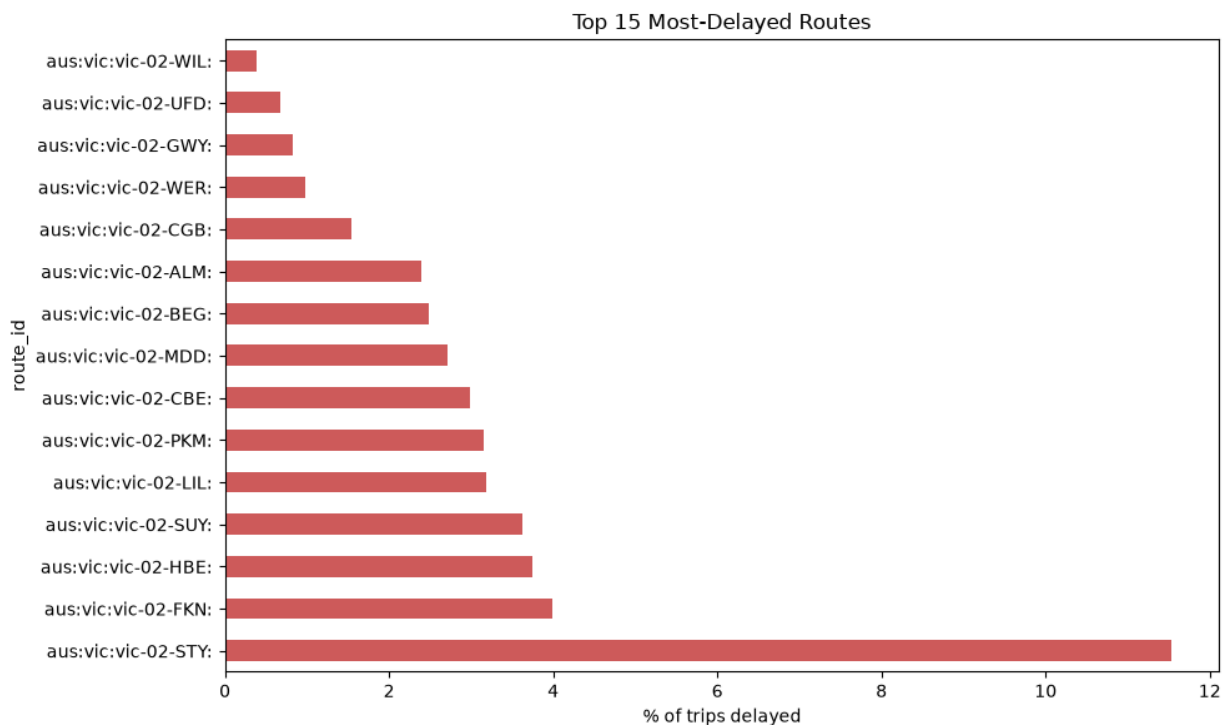


Figure 1- Most delayed routes

Based on figure 1 delays varied significantly by route ranging from under 1% to 11%. The Stony Point Line stood out as an outlier having the highest delay rate of around 11% while the second highest only had around 4%. This can be explained by Stony Point Line's single diesel track and its lower service frequency compared to the other lines.

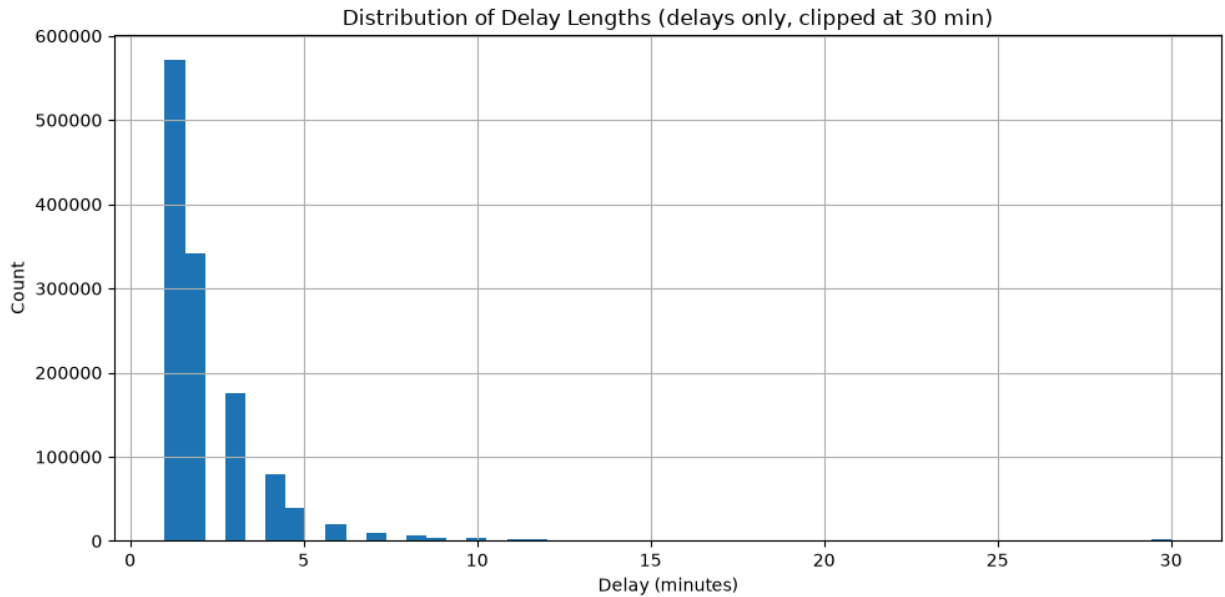


Figure 2 - Distribution of Delays

The distribution of delays is heavily rightskewed shown from figure 2 with the majority of delays being between 1 and 5 minutes with an exponentially decreasing frequency of delays as delay times get longer. This confirms that delays in this dataset are predominately minor disruptions rather than longer interruptions to services, this supports the use of a binary delay classification throughout this project.

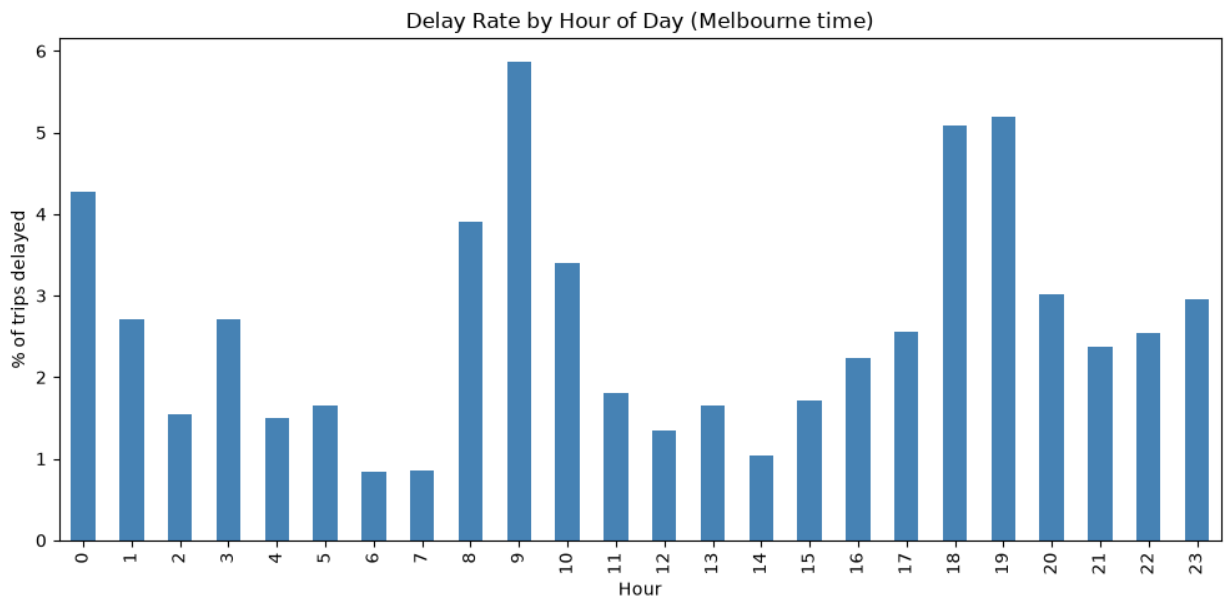


Figure 3 - Delay Rate by Hour

Delays show a clear relationship with the time of day. Shown in figure 3 there are clear spikes in delays during 8-10am in the morning and 6-7pm in the evening. This aligns with expected congestion patterns due to peak time travel.

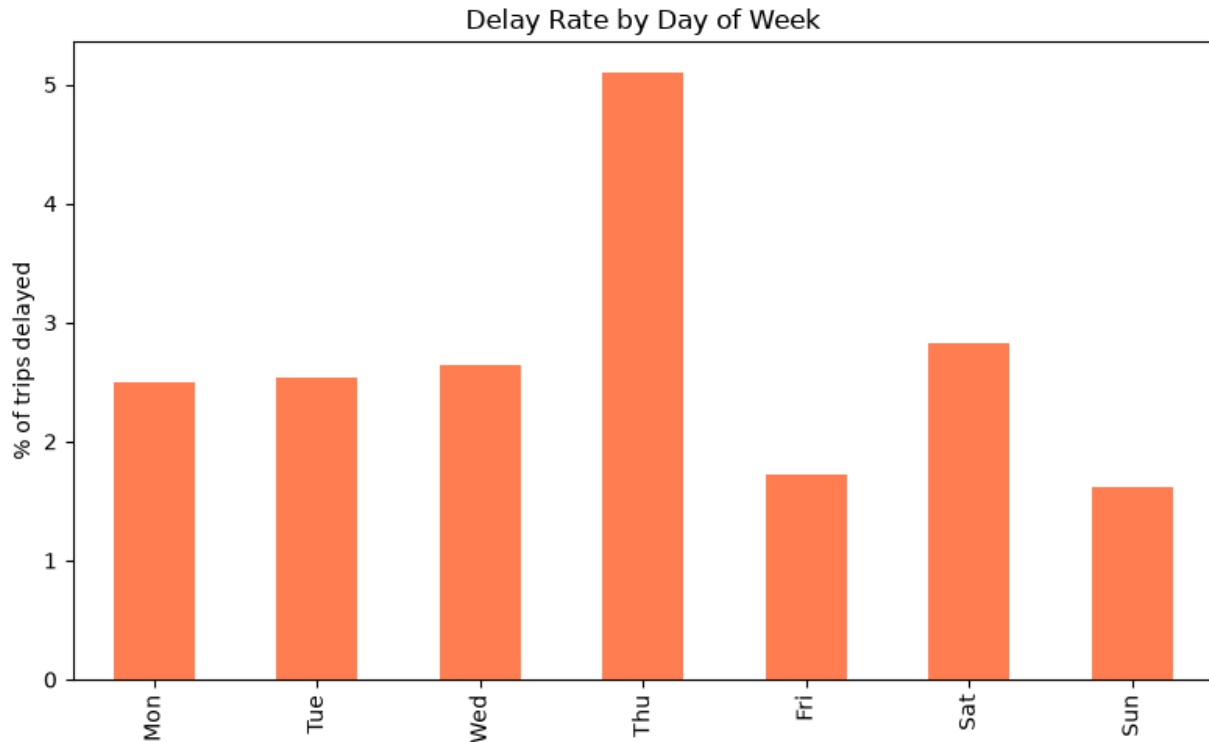


Figure 4 - Delay rate by day

Delays rates by data of the week remained relatively consistent with only Thursday showing a notably higher rate of around 5% compared to the other days being around 3%. However, given this dataset spans 14 days and thus only two occurrences of each weekday, this is likely to represent one particularly disruptive event on a Thursday rather than a genuine pattern.

Data Preprocessing-

Based on issues identified during exploratory analysis, several preprocessing steps were applied to the dataset before modelling.

Delay outliers- Records with unrealistic delay values that were identified during EDA (explanatory data analysis) were removed restricting `delayed_minutes` to a range of -30 to 120 minutes.

Missing weather data- A small number of records containing missing temperature ,windspeed ,precipitation and weather code data. These were imputed using the median of their respective column to avoid unnecessarily removing data

Engineering time features- `is_peak_hour` flags whether a trip occurred in the morning (7-9am) or evening (4-6pm) of a weekday which were identified as high delay areas during EDA, and `is_weekend` was created to flag Saturday and Sundays.

Category Encoding- Route and stop identifiers were originally strings and were converted to integer codes for use in machine learning. The category mapping was saved and consistently implemented across all stages of the pipeline to ensure a route was given the same code regardless of what data was present at that stage.

Timezone Correction- Originally due to our data collection service being implemented on Railway the time was recorded in UTC which meant that the hour and day did not reflect Melbourne time. This was fixed with all timestamps being recalculated to the corresponding Melbourne time. Without this correction all patterns would have been shifted by 10 hours resulting in misleading results.

A key engineered feature, `recent_route_delay` was constructed to capture persistent delays in that route. “`recent_route_delay`” averages on a given route the delays for the last hour excluding the trip's own delay. This feature was created due to observations on how delays persist and affect future delays rather than being isolated independent events.

On initial implementation this feature contained a data leakage bug. Due to the dataset having multiple records for a train for the same stop, `recent_route_delay` would capture the trains current delay rather than historic data. This issue was identified by an unusual high model performance and through inspection of `trip_id` values revealed consecutive observations of the same train. To remove the bug the data was aggregated to a unique observations per collection interval and recalculated. After this change to `recent_route_delay` model performance returned to realistic levels.

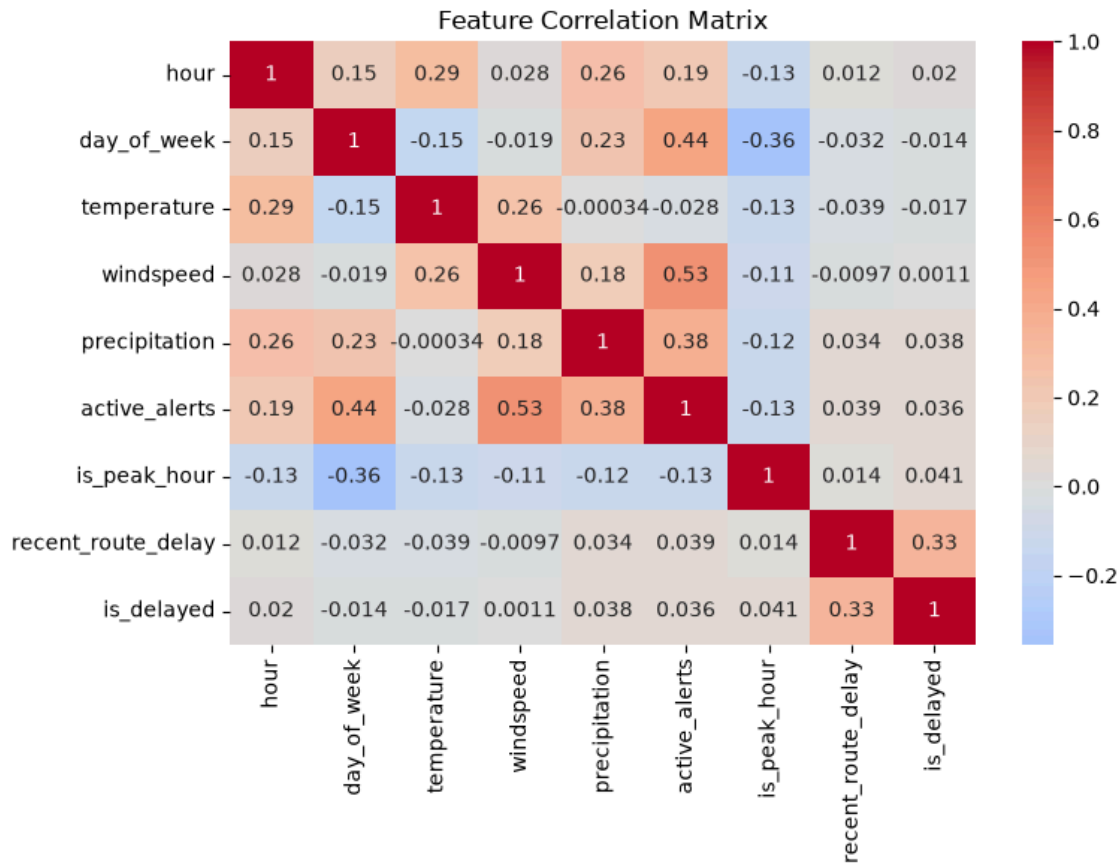


Figure 5 - Feature Correlation Matrix

Figure 5 presents the Pearson correlation matrix for the features used in machine learning and the target variable (`is_delayed`). Most features showed a weak linear correlation with `is_delayed`, indicating no single feature had a strong relationship with delay occurrence. The strongest correlation with `is_delayed` was observed for `recent_route_delay` ($r=0.33$), suggesting that recent delays on a route were significant for predicting future delays. Some moderate correlations were observed with other predictor features, such as `active_alerts` and wind speeds ($r=0.53$) and `active_alerts` and `day_of_the_week` ($r=0.44$). However, these correlations describe relationships between predictor variables rather than directly being correlated with delay occurrence.

Machine Learning-

Four models were trained and compared, Logistic Regression, Random Forest, XGBoost and LightGBM. These models were chosen due to them utilising differing classification methods. Logistic regression acts as a simple linear baseline, Random Forest allows us to see how decision tree based classifications perform and XGBoost and LightGBM allows us to see how gradient boosting performs.

The dataset spans a continuous period of 14 days therefore applying a standard random train/test data split approach would not be appropriate. Randomly splitting the data could leak

delays from records after or during the same train journey, leading to overly optimistic model performance. Instead, TimeSeriesSplit was used to create 5 chronologically ordered folds. Each fold was trained using all the observations before it and then evaluated on that period. This more closely reflects real world application where only past data is known.

The dataset was split chronologically into training, validation and test sets. Model hyper parameterization and threshold selection were performed exclusively using the validation set. Each model was then trained using a combined training and validation set before being evaluated by the untouched test set. This structure was adopted after earlier implementations where decision thresholds were selected using the test data leading to optimistic results.

Because only 2.63% of records were delayed, there was a need to address class imbalance during model development. For Logistic Regression and Random Forest different class_weight values were evaluated, while XGBoost and LightGBM tested a range of weight values with the optimal weighing being selected on performance on the validation set.

Furthermore, due to the class imbalance, the default classification threshold of 0.5 would skew the models toward non delayed due to it being larger class and therefore less risky. To mitigate this for each model a precision recall curve was used to identify the threshold that would maximise F1 score.

Finally, model performance was accessed using a five fold TimeSeriesSplit cross validation. This was done to assess how consistent each model performed rather than relying on a single split, it also provided insight into how model performance changed as more data became available. For each fold precision, recall, F1-score and ROC-AUC were calculated, with the mean and standard deviation reported to evaluate performance consistency.

Results-

Model performance was evaluated on an independent test set using five fold TimeSeriesSplit cross validation. Performance was assessed using precision, recall, F1 score and ROC-AUC.

Model	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.247 ± 0.122	0.409 ± 0.302	0.232 ± 0.136	0.794 ± 0.065
LightGBM	0.201 ± 0.078	0.306 ± 0.149	0.205 ± 0.097	0.800 ± 0.058
Random Forest	0.164 ± 0.060	0.217 ± 0.203	0.167 ± 0.109	0.735 ± 0.094
XGBoost	0.251 ± 0.110	0.309 ± 0.194	0.233 ± 0.116	0.825 ± 0.047

Table 1: Cross Validation Results

Table 1 summarises mean performance of each model across the five TimeSeriesSplit folds. XGBoost achieved the highest average F1 score (0.233 ± 0.116) whilst logistic regression achieved a similar F1 score (0.232 ± 0.136). The relatively large standard deviation indicates

model performance varied considerably across different time periods. LightGBM and Random Forest produced lower average F1 scores; however, all models performed relatively similarly.

Model	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.402	0.587	0.477	0.884
LightGBM	0.322	0.543	0.404	0.893
RandomForest	0.286	0.615	0.391	0.864
XGBoost	0.307	0.510	0.383	0.898

Table 2: Final Model Results

After hyperparameter tuning and optimal threshold selection each model was trained using the validation and training sets combined before being evaluated on the test set (Table 2). Logistic regression received the highest F1 (0.477) and precision (0.402) scores. XGBoost had the highest ROC-AUC score despite having the lowest F1 score. Random Forest had the highest recall(0.615).

Class	Precision	Recall	F1 Score	Support
Non delayed	0.99	0.97	0.98	362764
Delayed	0.40	0.59	0.48	12654
Accuracy			0.96	375418
Macro avg	0.69	0.78	0.73	375418
Weighted avg	0.97	0.96	0.96	375418

Table 3: Logistic regression results (Threshold 0.82)

Table 3 presents the classification report for Logistic Regression, which was selected due to it achieving the highest F1 score. The model correctly identified 59% of all delayed services while having an precision of 40%.

Model	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.047	0.349	0.083	0.610
LightGBM	0.065	0.101	0.079	0.643
RandomForest	0.142	0.092	0.111	0.737
XGBoost	0.057	0.258	0.094	0.677

Table 4: Performance without recent delay feature

Recent_route_delay was identified in the Pearson correlation matrix to have the highest correlation with delays. To evaluate this feature's effect the models were retrained without recent_route_delay. Performance overall declined significantly. Logistic regression experienced the biggest reduction from 0.477 to 0.083 with other models being similarly affected. RandomForest performed the best, having the largest F1 score (0.111) ; however, these results indicate that recent delay information is a highly important predictor for delays.

Discussion-

The aim of this project was to investigate whether short term PTV train delays could be predicted using publicly available metro, weather and train alert data. The results indicate that short term delays can be predicted with moderate success utilising available features. All four models achieved ROC-AUC values above 0.864 indicating they were reasonably able to distinguish delayed and non delayed services.

Logistic regression achieved the highest F1 score (0.477) on the final test set whilst, XGBoost achieved the highest ROC-AUC (0.898) and highest average F1 score across cross validation folds. Despite this, Logistic Regression was selected as the best performing model because ROC-AUC measures a models ability to distinguish between delayed and non delayed services across all classification thresholds, whilst F1 scores measure performance at a selected threshold. As the aim of the project focuses on identifying delayed services rather than risk assessing, F1 score is more valuable in terms of performance due to it being more relevant.

A significant finding was the impact that the feature recent_route_delays had. After this feature was removed the F1 score of Logistic Regression decreased from 0.477 to 0.083 with similar decreases across all models (Table 4). Although the models still performed better than random guessing, having F1 scores of 0.079 to 0.111 and ROC-AUC scores of 0.61 to 0.74, this is a substantial drop in predictive power. This indicates that weather,time and route features provide substantially less predictive than recent delays history. This suggests that the model is more suited for short term delay detection rather than forecasting further future delays.

Model performance varied notably across the five TimeSeriesSplit folds, with all models demonstrating improved performance in later folds. For example XGBoost's F1 score increased from 0.086 to 0.437 across its first and final fold, with its ROC-AUC increasing from 0.775 to 0.902. This pattern was shown in all models with XGBoost and LightGBM having a relatively consistent upward trend whilst Random Forests was more unstable. This consistent improvement as more training data became available indicates that the limited training window may be a key constraint on model performance.

Limitations-

The dataset used in this project was collected across a continuous 14 day period, collecting approximately 2.25 million records. However, this resulted in only two occurrences of each individual weekday and limited the model's exposure to differing weather conditions and weekly patterns needed for classification. This can be seen in Figure 4 with the spike in delays on

Thursday likely being an outlier rather than a stable trend. As a result, model performance under different periods, weather conditions and seasons outside this timeframe are uncertain. Furthermore, model performance increased as the amount of available data increased under cross validation folds. This suggests that the 14 day period is not sufficient for models to fully learn underlying patterns and likely collecting over a longer period would produce more substantial results.

Delay was modelled as a binary classification problem whether a service was delayed more than a five minute threshold. This overlooks important information about the severity of delays, meaning that the model is unable to distinguish a minor delay and severe delay.

The Pearson's correlation matrix used to identify relationships between features however, this only captures linear relationships therefore any non-linear relationships would be undetected. As a result some features that may have appeared to be weakly correlated may have had predictive value through a non-linear pattern.

Conclusions-

This project aims to investigate whether short term PTV delays can be predicted with publicly available GTFS, weather and service alert data. The findings show moderate model success with the final logistic regression model having an F1 score of 0.477 and ROC-AUC score of 0.884. These results indicate short term delay prediction is possible despite the severe class imbalance in the data set.

The most significant finding was recent_route_delay's feature strength in correlation to predicting delays. Removal of this feature showed significant decreases in F1 score and ROC-AUC across all models, this indicates that model predictions heavily focus on extrapolating delays short term rather than predicting future delays. Therefore, this model is better suited to determining if delays will persist over a short period rather than longer term future predictions. This narrows predictive scope, restricting the model to short term detection. However, results showed consistent improvement across all models during cross validation where more training data was available, suggesting that this may be addressed by extending the data collection period and amount of training data.

Future research should extend data collection periods to capture seasonal variations and to reinforce existing patterns with a larger sample. Research could also explore how far in advance delays can be predicted through weather forecasting, trends and known scheduled events. Furthermore, a regression model approach could be implemented providing insight into how severe delays are as well as offering better understanding to the limits and stability of delay predictions. Additionally, utilising a route specific alert system rather than a total service alert count may help provide targeted information on which routes are affected rather than treating all routes equally.

